

SYSTEM ANALYSIS & DESIGN

Adaptive Authentication with Behavioral Biometrics

Graduation Project — Technical Documentation

Academic Year 2025 - 2026

Document Type	System Analysis and Design Report
Project Title	Adaptive Authentication with Behavioral Biometrics
Scope	Web Application Security Layer
Technologies	.Net / Scikit-learn / JavaScript / React / Microsoft SQL
Status	Final Submission

This document constitutes the full system analysis and design specification for the Adaptive Authentication graduation project. It covers problem definition, system modelling, database design, data flow analysis, UI/UX guidelines, deployment strategy, and API documentation in accordance with academic software engineering standards.

Table of Contents

1. System Analysis and Design

- 1.1 Problem Statement and Objectives
- 1.2 Use Case Diagram and Descriptions
- 1.3 Functional and Non-Functional Requirements
- 1.4 Software Architecture

2. Database Design and Data Modelling

- 2.1 ER Diagram
- 2.2 Logical and Physical Schema

3. Data Flow and System Behavior

- 3.1 Data Flow Diagram (DFD)
- 3.2 Sequence Diagrams
- 3.3 Activity Diagram
- 3.4 State Diagram
- 3.5 Class Diagram

4. UI/UX Design and Prototyping

- 4.1 Wireframes and Mockups
- 4.2 UI/UX Guidelines

5. System Deployment and Integration

- 5.1 Technology Stack
- 5.2 Deployment Diagram
- 5.3 Component Diagram

6. Additional Deliverables

- 6.1 API Documentation
- 6.2 Testing and Validation
- 6.3 Deployment Strategy

1. System Analysis and Design

1.1 Problem Statement and Objectives

Contemporary authentication mechanisms rely predominantly on static credentials — usernames and passwords — which are inherently vulnerable to phishing, credential stuffing, and brute-force attacks. Even multi-factor authentication (MFA) solutions based on SMS OTPs or hardware tokens introduce friction and can be circumvented through SIM-swapping and social engineering. The fundamental weakness lies in the binary nature of authentication: once a user is verified at login, the session is considered trusted until it expires, regardless of subsequent behavioral deviations.

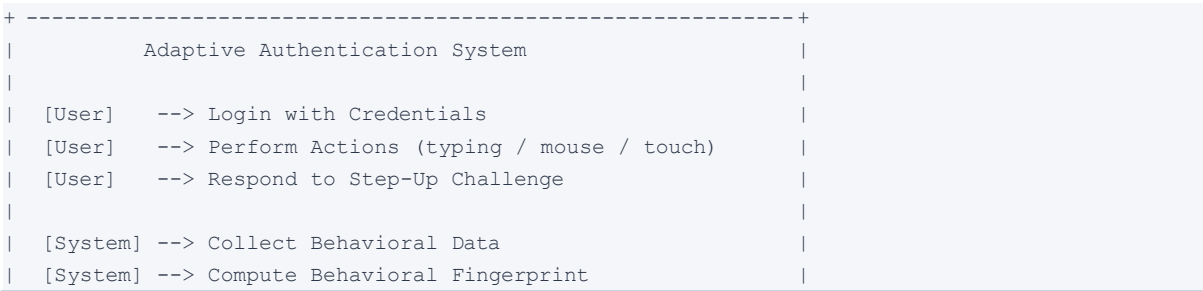
This project addresses the gap by designing and implementing an **Adaptive Authentication System** that continuously evaluates a user's behavioral biometric patterns throughout an active session. The system constructs a unique behavioral fingerprint for each user based on typing dynamics, mouse movement kinematics, and touch interaction patterns. Deviations from this fingerprint trigger risk-proportionate responses ranging from step-up authentication challenges to immediate session termination.

Project Objectives

- Develop a continuous, passive authentication layer that operates transparently alongside existing credential-based login systems.
- Construct individualized behavioral fingerprints using machine learning models (One-Class SVM, Autoencoders) trained on per-user interaction data.
- Implement real-time anomaly detection capable of scoring behavioral deviations and triggering appropriate security responses.
- Design a RESTful backend service deployable as a standalone security microservice or embeddable mobile SDK.
- Provide a transparent audit trail of all authentication events and security decisions for compliance and forensic purposes.
- Maintain a frictionless user experience for legitimate users while significantly raising the barrier for unauthorized access.

1.2 Use Case Diagram and Descriptions

The following use case diagram identifies the primary actors interacting with the Adaptive Authentication System and the system-level operations they initiate or trigger.



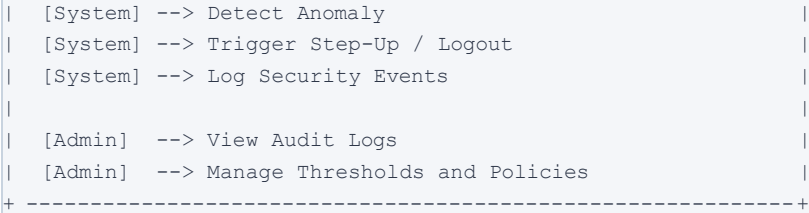


Figure 1 - Use Case Diagram: Adaptive Authentication System

Use Case	Actor	Description	Trigger	Outcome
Login with Credentials	User	Standard username/password authentication	User visits login page	Session token issued; behavioral collection begins
Behavioral Data Collection	System	Passive capture of typing, mouse, and touch events	User interacts with UI	Raw data buffered for feature extraction
Compute Behavioral Fingerprint	ML Engine	Feature engineering and model inference per session	Data buffer threshold reached	Anomaly score generated
Detect Anomaly	ML Engine	Comparison of live score against trained user profile	New score computed	Risk level updated: LOW / MEDIUM / HIGH
Trigger Step-Up Auth	System	Presents additional verification (OTP, CAPTCHA) to user	Risk level = HIGH	Session retained or terminated
Auto Session Logout	System	Terminates session and notifies the user	Failed step-up or critical anomaly	Session destroyed; security event logged
View Audit Logs	Admin	Access chronological record of security events	Admin requests report	Paginated event log returned
Manage Policies	Admin	Adjust anomaly thresholds and verification rules	Admin modifies config	System behavior updated in real-time

1.3 Functional and Non-Functional Requirements

Functional Requirements

ID	Requirement	Description
FR-01	User Authentication	The system SHALL authenticate users via username/password before behavioral monitoring begins.
FR-02	Behavior Collection	The system SHALL continuously collect keystroke timing, mouse velocity, and touch pressure data during active sessions.

FR-03	Feature Extraction	The system SHALL extract a fixed-length feature vector from raw behavioral data at configurable intervals (default: 30 s).
FR-04	Profile Training	The system SHALL train a per-user One-Class SVM or Autoencoder model upon accumulation of sufficient baseline data (min 50 samples).
FR-05	Anomaly Scoring	The system SHALL assign a continuous anomaly score [0.0 to 1.0] to each feature vector relative to the user's established profile.
FR-06	Risk Classification	The system SHALL classify sessions as LOW (below 0.4), MEDIUM (0.4 to 0.7), or HIGH (above 0.7) based on anomaly scores.
FR-07	Step-Up Challenge	The system SHALL present a secondary verification challenge when risk level reaches HIGH.
FR-08	Session Termination	The system SHALL automatically terminate sessions when step-up challenges are failed or risk remains critical.
FR-09	Audit Logging	The system SHALL persist all authentication events, anomaly scores, and security decisions with timestamps.
FR-10	Admin Dashboard	The system SHALL expose an administrative interface for reviewing audit logs and configuring detection thresholds.

Non-Functional Requirements

ID	Category	Description
NFR-01	Performance	Behavioral scoring SHALL complete within 200 ms per cycle; login SHALL complete within 500 ms at the 95th percentile.
NFR-02	Scalability	The system SHALL support horizontal scaling to handle 10,000 concurrent sessions without degradation.
NFR-03	Availability	The system SHALL achieve 99.9% uptime with automated failover and health checks.
NFR-04	Security	All behavioral data SHALL be encrypted at rest (AES-256) and in transit (TLS 1.3). Passwords SHALL use bcrypt with cost factor 12 or higher.
NFR-05	Privacy	Raw behavioral data SHALL be anonymized within 24 hours; only derived feature vectors SHALL be retained for model training.
NFR-06	Usability	The system SHALL be transparent to legitimate users; step-up challenges SHALL occur fewer than 1% of sessions for enrolled users.
NFR-07	Accuracy	The anomaly detection model SHALL achieve a False Positive Rate (FPR) below 2% and True Positive Rate (TPR) above 95% on validation sets.
NFR-08	Maintainability	The system SHALL follow a microservices architecture enabling independent deployment and testing of individual components.

1.4 Software Architecture

The system adopts a **Microservices Architecture** organized into discrete, independently deployable services that communicate via RESTful HTTP APIs and an internal message queue. This architectural style was selected to allow independent scaling of the computationally intensive ML Engine without affecting the availability of the Auth Service or Frontend.

Architectural Layer	Description
Presentation Layer	React.js SPA hosted on CDN; Mobile SDK (JavaScript/React Native). Communicates with the API Gateway via HTTPS.
API Gateway	Nginx reverse proxy routing authenticated requests to backend microservices; enforces rate limiting, WAF rules, and SSL termination.
Auth Service	FastAPI (.Net) service managing user registration, credential validation, JWT issuance, and session lifecycle management.
Behavior Service	FastAPI (.Net) service receiving behavioral event streams, buffering raw data, invoking the ML Engine, and updating risk levels.
ML Engine	Scikit-learn-based service exposing scoring and retraining endpoints; manages per-user model persistence in object storage (S3/Azure Blob).
Admin Service	FastAPI service exposing audit log queries and policy management endpoints; restricted to admin roles via RBAC.
Data Layer	MicrosoftSQL (relational): users, sessions, behavioral logs, security events. Redis (cache): session tokens, real-time risk scores.

2. Database Design and Data Modelling

2.1 Entity-Relationship Diagram (ERD)

The ERD below depicts the five primary entities of the system's relational data model, their attributes, and the cardinality of their inter-entity relationships.

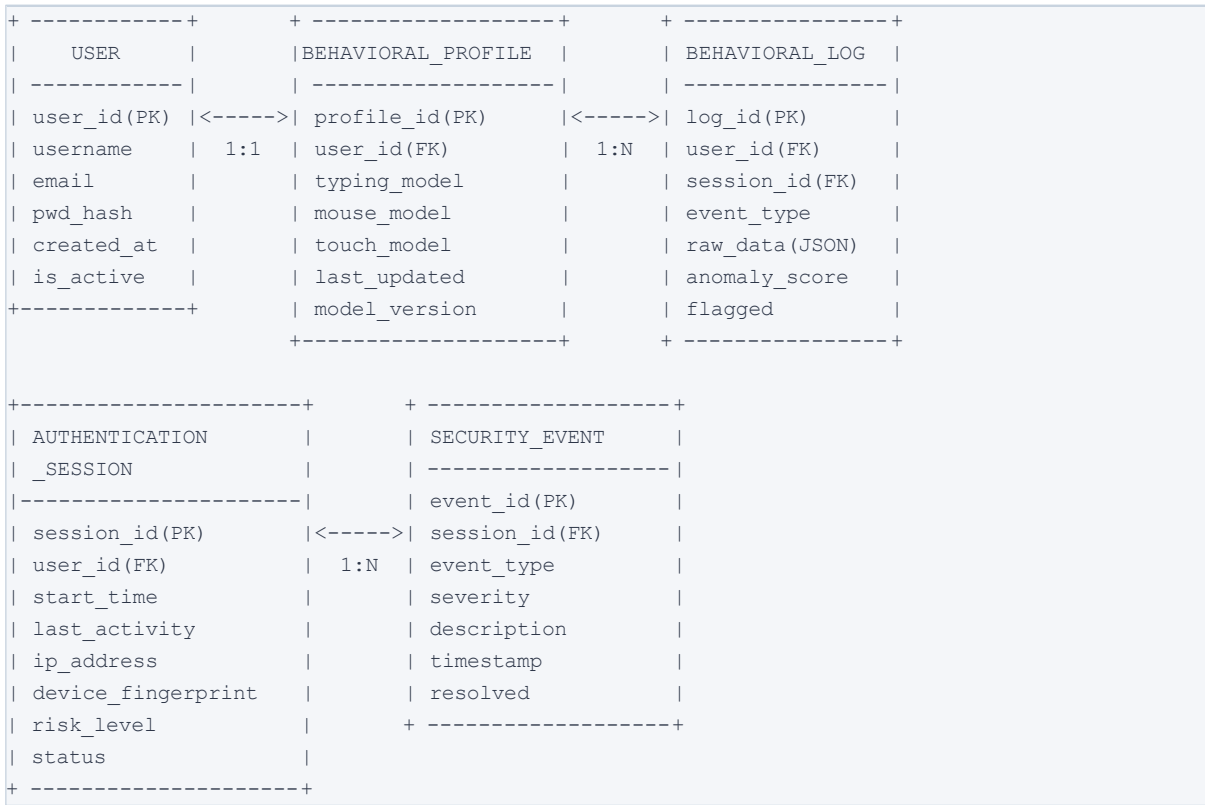


Figure 2 - Entity-Relationship Diagram

Entity	Description
USER	Stores registered user credentials and account metadata. Central entity with relationships to all other tables.
BEHAVIORAL_PROFILE	Maintains serialized ML model objects (One-Class SVM / Autoencoder) for each user, representing their behavioral fingerprint.
BEHAVIORAL_LOG	Append-only log of behavioral event records, raw feature vectors, computed anomaly scores, and flagging status.
AUTHENTICATION_SESSION	Tracks active and historical login sessions including device fingerprints, IP addresses, and real-time risk levels.
SECURITY_EVENT	Records security-relevant occurrences (step-up triggers, session terminations, policy violations) linked to sessions.

2.2 Logical and Physical Schema

The following DDL-style schema defines all tables, columns, data types, primary keys, foreign keys, and indexes. The schema is normalized to Third Normal Form (3NF) to eliminate redundancy and preserve referential integrity.

```
-- TABLE: users
CREATE TABLE users (
    user_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username       VARCHAR(50)  NOT NULL UNIQUE,
    email          VARCHAR(255) NOT NULL UNIQUE,
    password_hash  VARCHAR(255) NOT NULL,
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    is_active      BOOLEAN      NOT NULL DEFAULT TRUE
);

-- TABLE: behavioral_profiles
CREATE TABLE behavioral_profiles (
    profile_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id        UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    typing_model   BYTEA,
    mouse_model    BYTEA,
    touch_model    BYTEA,
    model_version  INTEGER NOT NULL DEFAULT 1,
    sample_count   INTEGER NOT NULL DEFAULT 0,
    last_updated   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id)
);

-- TABLE: authentication_sessions
CREATE TABLE authentication_sessions (
    session_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id        UUID NOT NULL REFERENCES users(user_id),
    started_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    last_activity   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    ip_address     INET,
    device_fingerprint VARCHAR(255),
    risk_level      VARCHAR(10) NOT NULL DEFAULT 'LOW'
                    CHECK (risk_level IN ('LOW','MEDIUM','HIGH')),
    status         VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
                    CHECK (status IN ('ACTIVE','CHALLENGED','TERMINATED','EXPIRED'))
);

-- TABLE: behavioral_logs
CREATE TABLE behavioral_logs (
    log_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id        UUID NOT NULL REFERENCES users(user_id),
    session_id     UUID NOT NULL REFERENCES authentication_sessions(session_id),
    event_type     VARCHAR(30) NOT NULL,
    raw_data       JSONB,
    feature_vector  DOUBLE PRECISION[],
    anomaly_score  DOUBLE PRECISION,
    flagged        BOOLEAN NOT NULL DEFAULT FALSE,
    recorded_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```



```
-- TABLE: security_events
CREATE TABLE security_events (
    event_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id    UUID NOT NULL REFERENCES authentication_sessions(session_id),
    event_type     VARCHAR(50) NOT NULL,
    severity      VARCHAR(10) NOT NULL CHECK (severity IN ('LOW','MEDIUM','HIGH','CRITICAL')),
    description    TEXT,
    occurred_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    resolved      BOOLEAN NOT NULL DEFAULT FALSE
);

-- INDEXES
CREATE INDEX idx_behavioral_logs_user      ON behavioral_logs (user_id);
CREATE INDEX idx_behavioral_logs_session  ON behavioral_logs (session_id);
CREATE INDEX idx_behavioral_logs_time     ON behavioral_logs (recorded_at DESC);
CREATE INDEX idx_sessions_user            ON authentication_sessions (user_id);
CREATE INDEX idx_security_events_session  ON security_events (session_id);
```

Listing 1 - PostgreSQL Physical Schema (3NF normalized)

3. Data Flow and System Behavior

3.1 Data Flow Diagram (DFD)

Context Level (Level 0)



Figure 3 - DFD Level 0: Context Diagram

Level 1 — Internal Processes

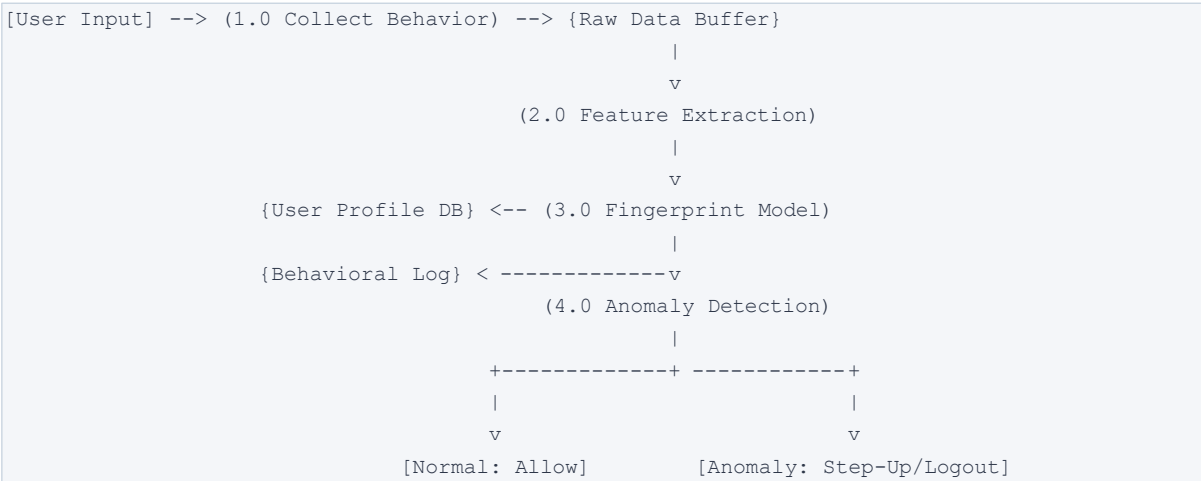
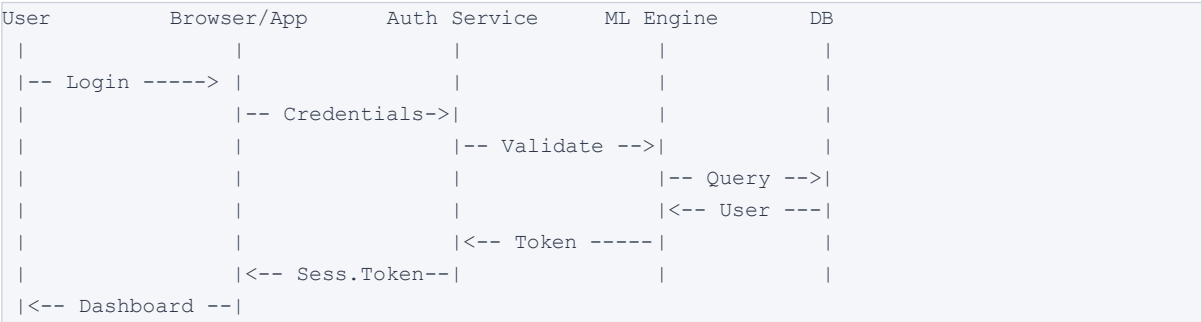


Figure 4 - DFD Level 1: Internal Data Flows

3.2 Sequence Diagrams

Login and Continuous Behavioral Authentication Flow



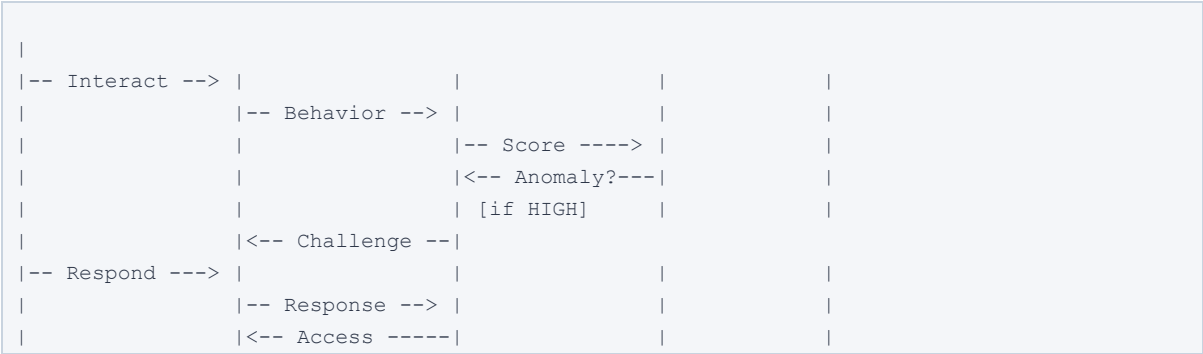


Figure 5 - Sequence Diagram: Login and Real-time Behavioral Scoring

3.3 Activity Diagram

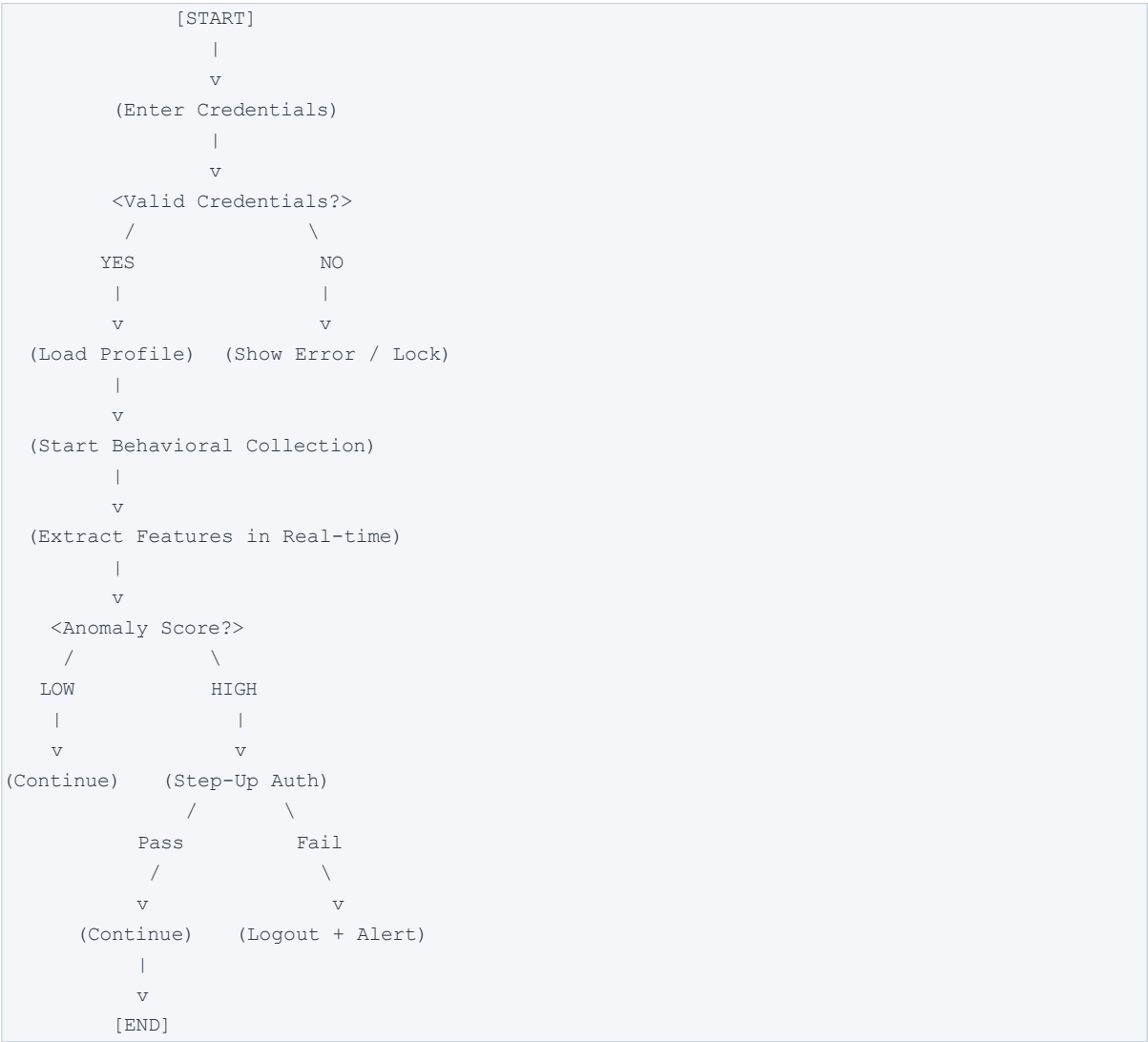


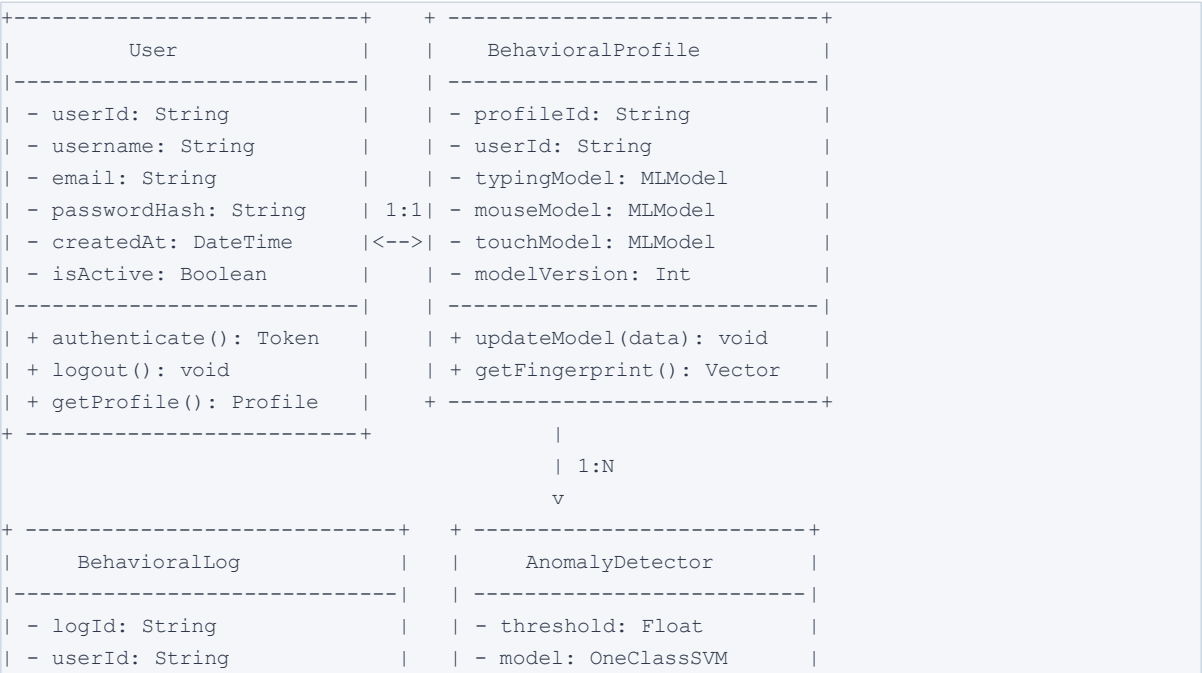
Figure 6 - Activity Diagram: Adaptive Authentication Workflow

3.4 State Diagram



Figure 7 - State Diagram: Session Lifecycle

3.5 Class Diagram



```
| - sessionId: String          | | ----- |
| - eventType: Enum           | | + score(vector): Float   |
| - rawData: JSON             | | + isAnomaly(s): Bool     |
| - anomalyScore: Float       | | + retrain(data): void    |
| - flagged: Boolean          | + -----+
| ----- |
| + save(): void              |
| + flag(): void              |
+ -----+
```

Figure 8 - Class Diagram: Core Domain Model

4. UI/UX Design and Prototyping

4.1 Wireframes and Mockups

The following ASCII wireframes illustrate the key screens of the system. High-fidelity mockups would be produced in Figma or Adobe XD following these structural layouts.

Screen 1 — Login Page

```
+-----+
| [LOGO] SecureAuth |
|
| Email: [ ] |
| Password: [ ] |
|
| [ Sign In ] |
|
| Forgot password? Create account |
+-----+
```

Wireframe 1 - Login Screen

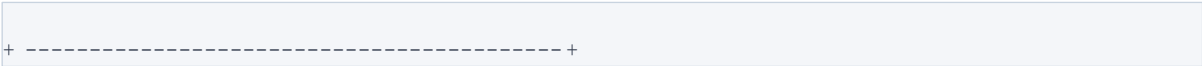
Screen 2 — Main Dashboard (Active Monitoring Indicator)

```
+-----+
| [Logo] Dashboard [User: John] [Logout] |
+-----+
| Security Status: [CIRCLE LOW RISK] |
|
| +-----+ +-----+ |
| | Recent Actions | | Behavioral Score | |
| | - Login 9AM | | Typing: [=====] 82% | |
| | - Edit doc | | Mouse: [=====] 90% | |
| | - Upload | | Touch: [===] 60% | |
| +-----+ +-----+ |
|
| [View Full History] [Account Settings] |
+-----+
```

Wireframe 2 - User Dashboard with Real-time Risk Indicator

Screen 3 — Step-Up Authentication Challenge

```
+-----+
| ! Security Verification Required |
|
| Unusual activity detected in your |
| session. Please verify your identity. |
|
| Enter the OTP sent to j***@gmail.com: |
|
| [ ] [ ] [ ] [ ] [ ] [ ] |
|
| [ Verify ] [Resend Code] |
+-----+
```



Wireframe 3 - Step-Up Authentication Challenge Modal

4.2 UI/UX Guidelines

Guideline	Specification
Color Palette	Primary: #2E5090 (Navy Blue) — trust and security. Accent: #4A90D9 (Sky Blue) — interactive elements. Alert: #E53E3E (Red) — high risk / errors. Success: #38A169 (Green) — low risk / confirmation. Neutral: #F4F6F9 (Light Grey) — backgrounds.
Typography	Headings: Inter Bold, 18-24px. Body: Inter Regular, 14-16px, line-height 1.6. Monospace (codes/logs): JetBrains Mono, 13px. Minimum contrast ratio: 4.5:1 (WCAG AA).
Layout & Grid	12-column grid, 24px gutters, 1280px max-width container. Responsive breakpoints: 375px (mobile), 768px (tablet), 1280px (desktop). Card-based layout with 12px border-radius.
Accessibility	WCAG 2.1 AA compliance. Full keyboard navigation support. ARIA labels on all interactive elements. Focus indicators: 2px solid #4A90D9. Screen reader compatibility tested.
Feedback & States	All interactive elements respond within 100ms. Skeleton loaders for data fetching above 300ms. Inline validation on form fields (real-time). Toast notifications for security events.
Security UX	Risk indicator always visible in navigation bar (color-coded). Step-up challenges presented as modal overlays. Session timeout warnings shown 2 minutes before expiry. All security events communicated in plain language.

5. System Deployment and Integration

5.1 Technology Stack

Layer	Technology
Frontend	React 18 (TypeScript), Tailwind CSS, Vite bundler, React Query for API state management.
Behavior SDK	Vanilla JavaScript event listeners (keydown, mousemove, touchstart); packaged as an NPM module for easy integration.
Backend	.Net 3.11, FastAPI, Pydantic v2 for data validation, SQLAlchemy 2.0 ORM.
ML / AI	Scikit-learn (One-Class SVM, Isolation Forest), PyTorch (Autoencoder variant), NumPy and Pandas for feature engineering.
Database	MicroSoftSQL 15 (primary relational store), Redis 7 (session cache and real-time risk scores).
Authentication	JWT (RS256 signed), bcrypt for password hashing, TOTP (pyotp) for step-up OTP generation.
Infrastructure	RDS (MicroSoftSQL), ElastiCache (Redis), S3 (model storage),.
Monitoring	Prometheus and Grafana (metrics), AWS CloudWatch (logs), Sentry (error tracking).

5.2 Deployment Diagram

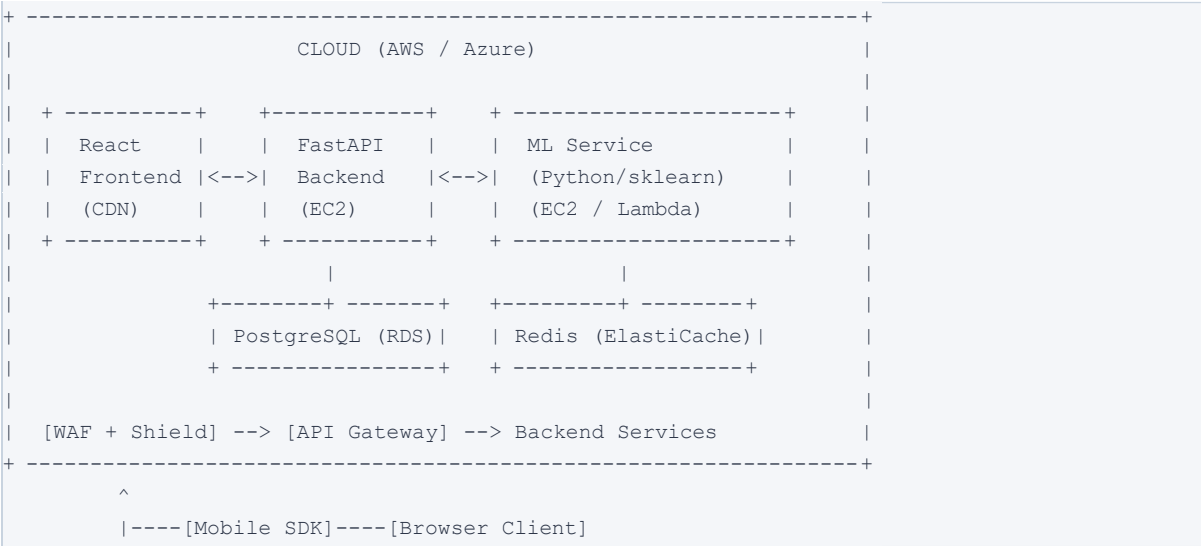


Figure 9 - Deployment Diagram: Cloud Infrastructure

5.3 Component Diagram

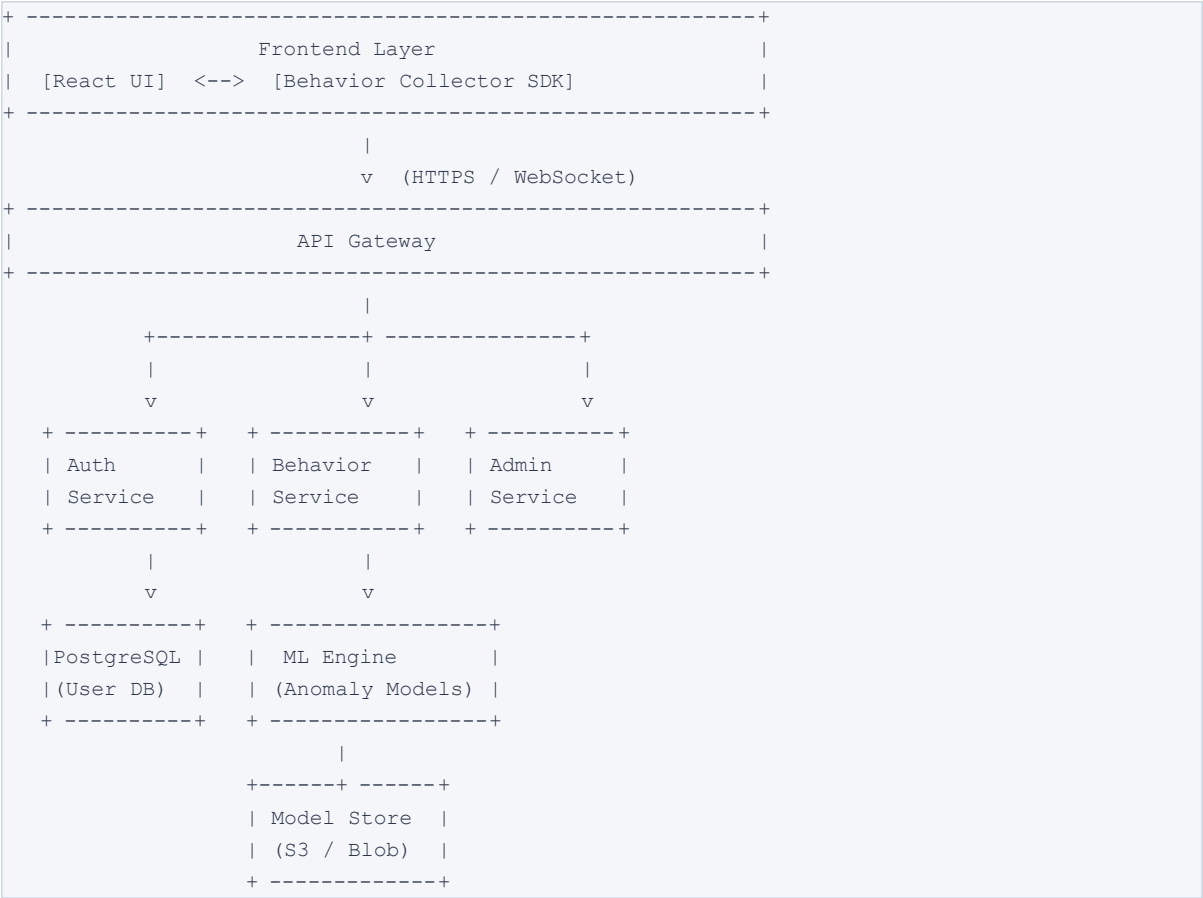


Figure 10 - Component Diagram: System Dependencies

6. Additional Deliverables

6.1 API Documentation

All backend services expose RESTful JSON APIs. The base URL for the Auth Service is **https://api.secureauth.io/v1**. All endpoints (except /auth/login and /auth/register) require a valid Bearer JWT token in the Authorization header.

Method	Endpoint	Auth	Description	Request Body	Response
POST	/auth/register	Public	Register a new user account	{ username, email, password }	201: { user_id, message }
POST	/auth/login	Public	Authenticate and receive session token	{ email, password }	200: { access_token, token_type, expires_in }
POST	/auth/logout	Auth	Invalidate the current session	—	200: { message }
POST	/behavior/record	Auth	Submit a behavioral event batch	{ session_id, events:[...] }	202: { queued: true }
GET	/behavior/risk	Auth	Retrieve current session risk score	—	200: { risk_level, anomaly_score }
POST	/auth/challenge/initiate	Auth	Trigger step-up challenge	{ method: 'otp' }	200: { challenge_id, expires_at }
POST	/auth/challenge/verify	Auth	Submit step-up challenge response	{ challenge_id, code }	200: { verified: true } / 401
GET	/admin/logs	Admin	Retrieve paginated security event log	?page=1&limit=50	200: { events:[...], total, page }
PUT	/admin/thresholds	Admin	Update anomaly detection thresholds	{ low_threshold, high_threshold }	200: { updated: true }

6.2 Testing and Validation Plan

Unit Testing

Unit tests are written using **pytest** (Python) and **Jest** (JavaScript). Each microservice targets a minimum of 80% code coverage. Critical modules — particularly the anomaly scoring pipeline and JWT validation logic — target 95% coverage.

ID	Component	Test Description
----	-----------	------------------

UT-01	Feature Extractor	Validates extraction of correct feature vector dimensionality and value ranges from synthetic input data.
UT-02	Anomaly Scorer	Verifies One-Class SVM returns scores within [0.0, 1.0] and correctly classifies known normal/anomalous samples.
UT-03	JWT Service	Tests token generation, signature verification, expiry enforcement, and revocation.
UT-04	Password Hashing	Confirms bcrypt hash/verify round-trips and rejects tampered hashes.
UT-05	Risk Classifier	Validates threshold-based classification logic against boundary values (0.39, 0.40, 0.70, 0.71).

Integration Testing

Integration tests validate inter-service communication using a Docker Compose environment with live PostgreSQL and Redis instances. Tests are executed as part of the GitHub Actions CI pipeline on every pull request.

ID	Test Case	Description
IT-01	End-to-End Login Flow	Registers user, logs in, receives token, verifies token validity against protected endpoint.
IT-02	Behavioral Pipeline	Sends synthetic behavioral event batch, verifies data persistence in behavioral_logs, confirms risk score update in Redis.
IT-03	Step-Up Challenge Flow	Triggers artificial HIGH risk state, verifies challenge issuance, simulates correct OTP submission, confirms session retention.
IT-04	Session Termination	Simulates failed step-up response, verifies session status set to TERMINATED and security event logged.
IT-05	Admin Log Retrieval	Creates 100 security events, queries admin /logs endpoint with pagination, verifies ordering and count.

User Acceptance Testing (UAT)

ID	Scenario	Acceptance Criterion
UAT-01	Legitimate User Frictionless Access	A legitimate user performs normal activities for 30 minutes; system MUST NOT trigger step-up challenge (FPR target below 2%).
UAT-02	Impostor Detection	A different individual uses a legitimate user's credentials; system MUST flag anomaly and challenge within 5 minutes.
UAT-03	Step-Up Verification Usability	Users MUST be able to complete OTP step-up challenge within 60 seconds on first attempt.
UAT-04	Admin Audit Log Usability	Admin MUST locate specific security event within 2 minutes using provided search and filter tools.

6.3 Deployment Strategy

Hosting Environment

Production deployment targets **Amazon Web Services (AWS)** in the eu-west-1 (Ireland) region, with a disaster recovery replica in eu-central-1 (Frankfurt). All resources are provisioned using Terraform for infrastructure-as-code consistency.

Deployment Pipeline (CI/CD)

Step	Action
1. Code Push	Developer pushes feature branch to GitHub; pull request opened targeting the main branch.
2. Automated Tests	GitHub Actions runs: unit tests (pytest + Jest), integration tests, static analysis (flake8, eslint), security scan (Bandit, npm audit).
3. Code Review	Peer review required; minimum 1 approval from a senior developer before merge is permitted.
4. Build & Push	On merge to main: Docker images built and pushed to Amazon ECR with semantic version tags.
5. Staging Deploy	ArgoCD auto-deploys to staging EKS cluster; smoke tests executed against staging endpoints.
6. Production Deploy	Manual approval gate in GitHub Actions; blue/green deployment to production EKS cluster via ArgoCD.
7. Post-Deploy	Automated health checks executed; Prometheus alerts configured; rollback triggered automatically if error rate exceeds 1% within 10 minutes.

Scaling Considerations

Concern	Strategy
Horizontal Scaling	All microservices are stateless and deployed as Kubernetes Deployments with HorizontalPodAutoscaler (HPA) policies based on CPU utilization (target: 70%).
ML Model Caching	Trained user models are cached in memory within ML Engine pod instances (LRU cache, max 500 models). Cold loads from S3 incur less than 200 ms latency.
Database Scaling	MicrosoftSQL read replicas (x2) for reporting queries; connection pooling via PgBouncer. Redis Cluster mode for session cache scalability.
CDN & Edge	React SPA assets served via CloudFront CDN; behavioral SDK delivered via edge locations to minimize initial load latency globally.

End of System Analysis and Design Document. Adaptive Authentication with Behavioral Biometrics. All diagrams, schemas, and specifications contained herein are subject to revision during the implementation phase. This document fulfills the academic deliverable requirements for the graduation project submission.